

OWASP Secure Coding Compliance Report

Evaluating Game Security via Academic Software Design Methodologies

Course Context:	CM2010 Software Design and Development
Target Project:	Blobby Adventure (Client-Side p5.js Platformer)
Source Files:	main.js, gameplay.js, entities.js, constants.js, generation.js
Reference Standards:	OWASP Secure Coding Practices Quick Reference Guide v2.1

Executive Summary: This report analyzes the security architecture of the client-side game project *Blobby Adventure*. Using the **OWASP Secure Coding Practices (SCP) Quick Reference Guide v2.1**, i select five security items relevant to the game's execution context. I then explain how the secure programming techniques and engineering principles tught in the **CM2010** course can be applied to ensure secure code.

1. Selection & Analysis of OWASP Secure Coding Items

Item 1: [12] Validate Data Range (Input Validation)

- **Relevance to Project:** The game relies heavily on procedural coordinate layouts and physics simulations. For example, in `generation.js`, the world width scale is dynamically randomized. Unvalidated variables (negative scores, out-of-bounds values, etc.) could cause various problems (e.g. corrupt state, infinite loops, etc.).
- **Verification Technique:** *Runtime Assertions & Defensive Validation & Unit Testing.* We can enforce constraints by placing assertions. Although JavaScript has no native runtime assert, a helper function check constraints. For testing compliance, unit testing should be used. Automated test cases should feed boundary values (such as minimum and maximum expected coordinate ranges) to coordinates and physics functions to ensure the execution logic behaves predictably and does not crash.

Item 2: [13] Validate Data Length (Input Validation)

- **Relevance to Project:** During procedural layout generation, the engine dynamically populates collections such as `state.platforms` or `state.worms`. Without strict validations, scaling parameters could exhaust available memory space or cause excessive rendering overhead.
- **Verification Technique:** *Defensiv Bounding & Unit Testing.* The codebase uses a defensive technique by bounding iterative random placement within attempt limits (e.g., `attempts < MAX_ATTEMPTS`). Compliance is checked through unit testing to verify that generated collection sizes conform to density constraints. Developers can write unit test suites that check that entity counts are bounded and do not grow exponentially under larger randomized world configurations.

Item 3: Restrict Operations to Buffer Bounds (Memory Management)

- **Relevance to Project:** JavaScript manages memory automatically, avoiding typical low-level C/C++ memory overflows. However, accessing invalid indices (e.g., referencing an index out of bounds) returns undefined. Dereferencing properties on an undefined element will throw a `TypeError`, crashing the game.
- **Verification Technique:** *Defensiv Boundary Checks & Unit Testing.* I could implement defensive checks at indexing boundaries to guarantee element existence before member access. To check compliance, unit tests must cover edge inputs: so writing test cases that query empty collections, verifying behavior when accessing elements past array boundaries and confirming that functions handle missing indices gracefully. If a lookup fails, simple checks can prevent the application from crashing.

Item 4: Centralize Input Validation Routines (Input Validation)

- **Relevance to Project:** *Blobby Adventure* generates game entities (platforms, canyons, collectibles, enemies) using separate coordinate layout rules. Validating dimensions and safety zones in each generation function increases coupling and duplication, which heightens the risk of omitting integrity checks.

- **Verification Technique:** *The Factory Pattern (Modules) & Static Analysis Code Reviews.* The project decouples object construction via the **Factory Pattern** using a centralized factory object in `entities.js` (applying module design and cohesion principles). Compliance with centralized validation is achieved by embedding verification rules directly into the factory methods. Compliance can be checked via **Code Reviews / Static Analysis** to verify that raw new constructor calls are prohibited outside `entities.js`, enforcing the centralized validation structure.

Item 5: Manage Risk of Using Third-Party Components (System Configuration)

- **Relevance to Project:** The game relies on the external framework library `p5.js`. Using outdated or modified third-party packages exposes the client to security risks, cross-site scripting (XSS), or remote code execution in the user's browser.
- **Verification Technique:** *Software Security Development Lifecycle (SDL) Inventory & SCA.* As covered in Microsoft SDL principles, secure projects must establish a complete inventory of external resources. Compliance is checked via automated **Software Composition Analysis (SCA)** and dependency vulnerability scanners (e.g., `npm audit`). Furthermore, we can use **Static Code Reviews**.

2. Secure Software Development Lifecycle (SDLC) Integration

Integrating secure programming practices into the development lifecycle ensures security is intrinsic to the software, rather than an afterthought. We can integrate these practices using three distinct phases of the Software Security Development Lifecycle:

1. **Static Analysis (Security Linting):** During the coding phase, automated static analysis tools (similar to Bandit for Python, but targeting JavaScript, such as ESLint with security plugins) should be run. These tools inspect source files to identify vulnerable coding patterns before compile or run time.
2. **Unit Testing and Test-Driven Development (TDD):** Unit testing should be utilized to construct a regression test suite. Following TDD, test assertions should be written to verify validation rules before functional logic is written,.
3. **Defensive Assertions & Runtime Monitoring:** Implement runtime assertion checks at boundary transitions to verify that variables remain within safe envelopes during live gameplay execution, preventing unhandled runtime exceptions from leaking details or crashing the browser environment.